Binary Multiplication - Partial Sum Approach

|M| : n-bits |Q| : n-bits |M×Q| : (n+n) = 2n bits

Multiplicand (M) & Multiplier (Q)

For every bit in Q (from right to left),

i) If the bit is 0, perform right shift once on the register's content.

ii) If the bit is 1, first add M with the most significant n-bits of the register's content then perform right shift once.

Ex- Multiplicand (M) : 1010
Multiplier (Q) : 0110

Right shifts : n
Additions : # 1s in Q

0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
+ 1 0 1 0
1 0 1 0 0 0 0 0
0 1 0 1 0 0 0 0
+ 1 0 1 0
1 1 1 1 0 0 0 0
0 1 1 1 1 0 0 0
0 0 1 1 1 1 0 0

Summary

- Basic Binary Multiplication
- Partial Sum approach in Binary Multiplication.

Binary Multiplication - Booth's Algorithm

- Multiplies two signed binary numbers in two's complement notation.
- Invented by Andrew Donald Booth in 1950.

Multiplier (Q): 0 0 1 1 1 1 1 0 $\rightarrow 2^{5+1} - 2^1 = 64-2$
$2^7\ 2^6\ 2^5\ 2^4\ 2^3\ 2^2\ 2^1\ 2^0$ $= 62$

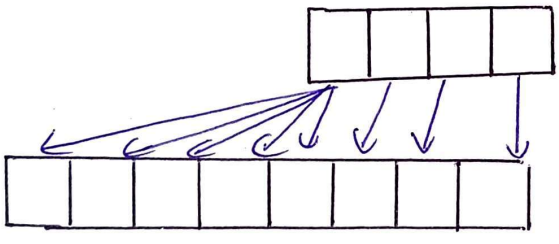
0 1 1 0 1 1 1 0 $\rightarrow (2^7 - 2^5) + (2^4 - 2^1)$
$2^7\ 2^6\ 2^5\ 2^4\ 2^3\ 2^2\ 2^1\ 2^0$ $= 96 + 14 = 110$

Generalization: 0 0 0 1 1 1 1 ..... 1 0 0 ... 0
(2^j ... 2^i)
$2^{j+1} - 2^i$

Multiplicand (M): 0 1 0 1 1 $(\overline{M})$: 1 0 1 0 1
Multiplier (Q): 0 1 1 1 0 $\rightarrow 2^4 - 2^1$
$2^4\ 2^3\ 2^2\ 2^1\ 2^0$

M x Q: $2^4(M) + 2^1(\overline{M})$

$2^4(M) = 01011 \times 2^4 = 0101 10000$
$2^1(\overline{M}) = 10101 \times 2^1 = 101010$



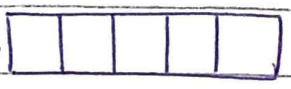
Copy the contents at the Least Significant Places & copy the MSB to the rest of the cells.

$2^1(\overline{M}) = 1111 01010$

$\therefore 2^4(M)$: 0 1 0 1 1 0 0 0 0 $\rightarrow$ 4 shifts
$2^1(\overline{M})$: 1 1 1 1 0 1 0 1 0 $\rightarrow$ 1 shift
} 5 shifts
1 0 1 0 0 1 1 0 1 0

By me

Implementation of Booth's Algorithm

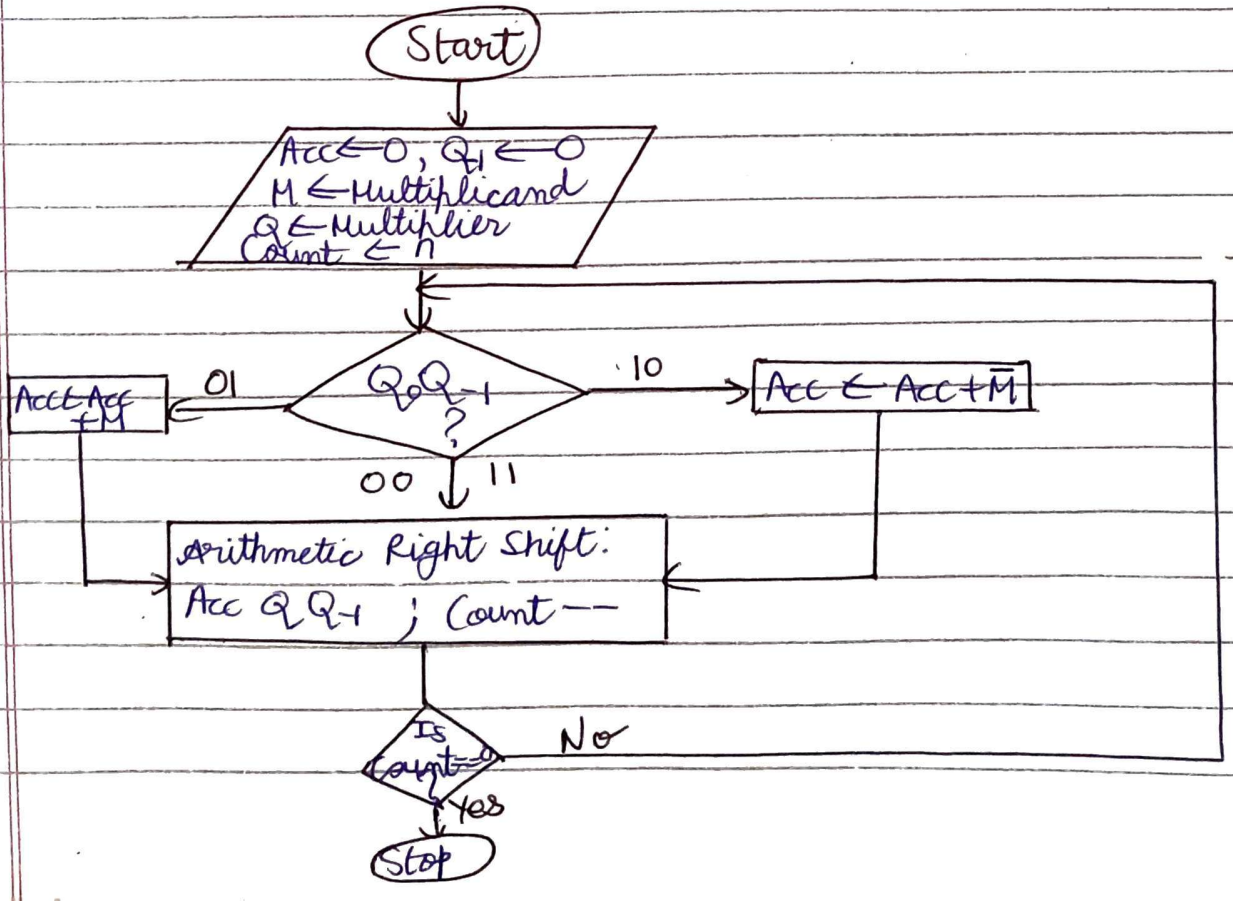
Multiplicand (M): 01011 → 

Multiplier (Q): 01110

MXQ: $2^4(M) + 2^1(\bar{M})$

Q_{LSB}	Q_{-1}	Operation
0	0	ARS
0	1	Acc ← Acc+M, ARS
1	0	Acc ← Acc+$\bar{M}$, ARS
1	1	ARS

Accumulator	Q	Q_{-1}
0 0 0 0 0	0 1 1 1 0	0
0 0 0 0 0	0 0 1 1 1	1 0
+1 0 1 0 1		
1 0 1 0 1	0 0 1 1 1	0
1 1 0 1 0	1 0 0 1 1	1
1 1 1 0 1	0 1 0 0 1	1
1 1 1 1 0	1 0 1 0 0	1
+0 1 0 1 1		
~~1~~ 0 1 0 0 1	1 0 1 0 0	1
0 0 1 0 0	1 1 0 1 0	0



PAGE NO
DATE:

Advantages

1. Performs signed multiplications.
2. Performs better if -ve numbers are involved
   e.g. 1111111111001 → -7
3. Reduced number of operations.

Disadvantages

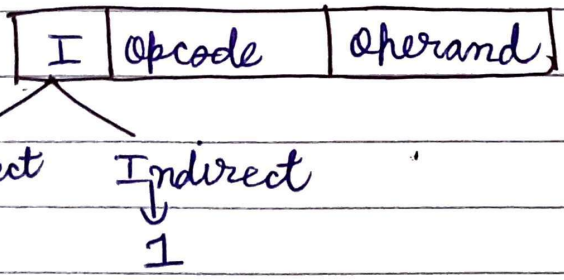
Performance decreases, if Q: 0101010101 ...

Summary

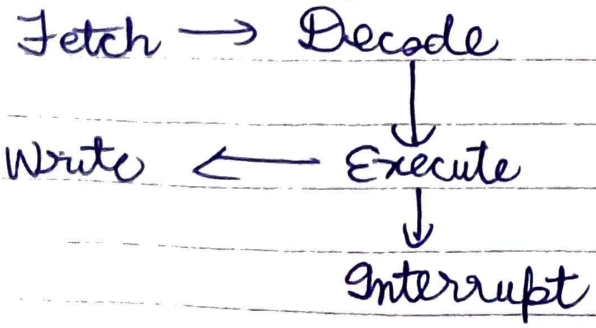
- ★ Implementation of Booth's Algo.
- ★ Advantages & Disadvantages

02/02/26

Instruction Register

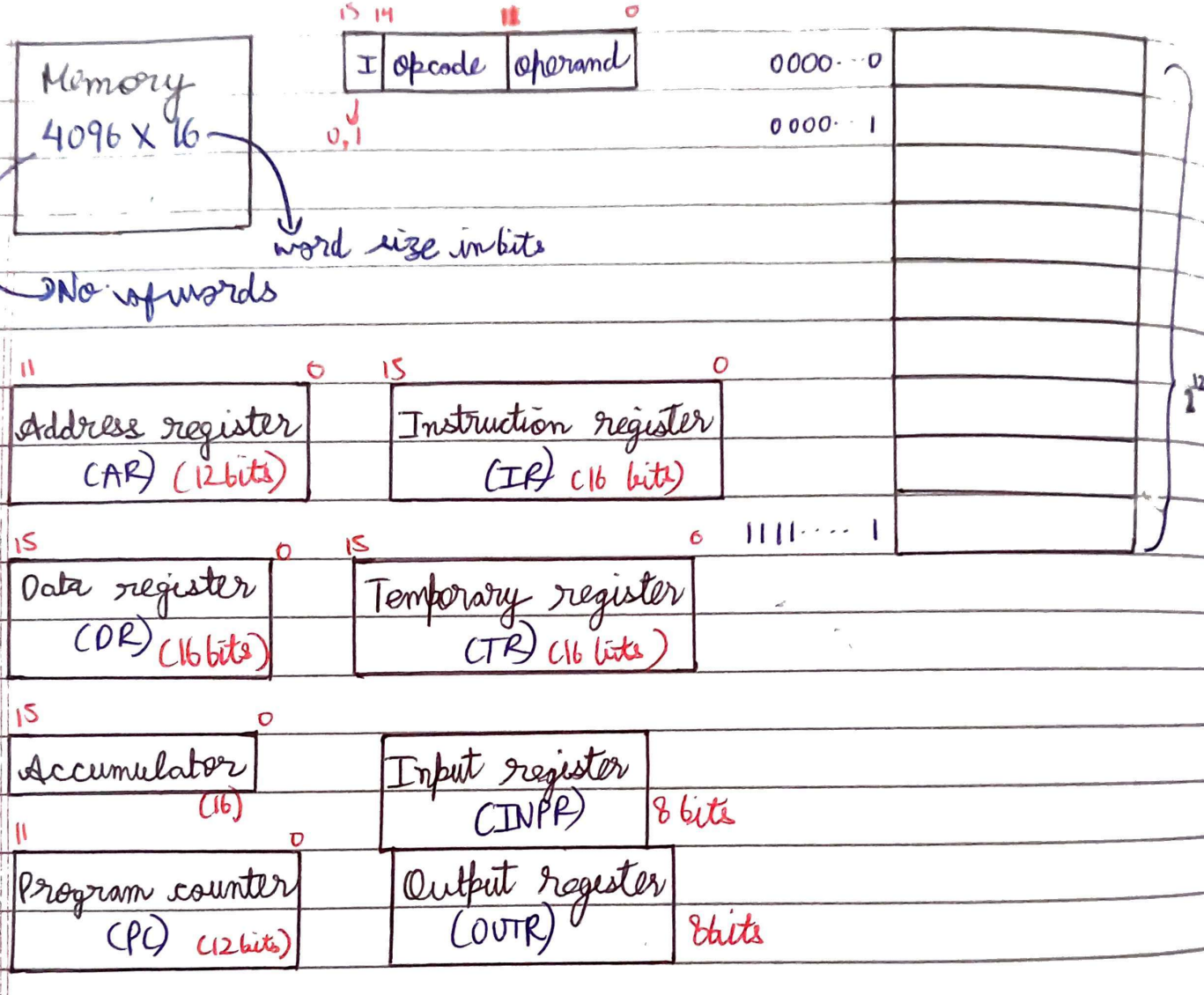


Instruction cycle



Crate Smashers

L-1·3



L-1·7

Types of Instructions

Data transfer instructions

- move
- store
- input
- push
- load
- exchange
- output
- pop

Data Manipulation Instructions

- Arithmetic
- Logical
- Shift/Conversion

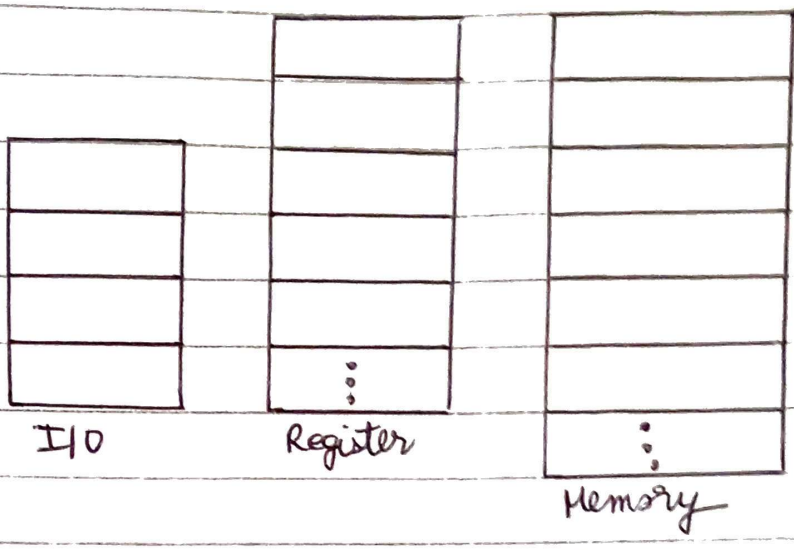
Program Control Instructions

- if, else if
- loop:
  - i) for ( )
  - ii) while ( )

Data Transfer Instructions

→ MOV

Eg. 1) MOV R1 R2
2) MOV R1 500
3) MOV R1 X
↳ memory location



→ Load : Loading data from memory to register

e.g. LD I 500 , LD R1, R2 , LD R1 X

→ Store: Storing data from register to memory.

e.g. STA

→ Exchange

Eg. XCHG R1 R2

→ Input → Output → Push → Pop

Arithmetic Instructions

Add
Sub
Mul
DIV
} Hardware available in ALU

INC
DEC
Add with Carry
Sub with borrow
Negate

PAGE NO.
DATE

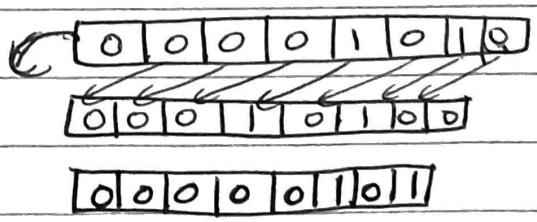
★ Multiple operations are being performed in backend to perform addition

Logical Instructions

→ Complement (COM or NOT)
→ Clear
→ Logical - AND (AND)
→ Logical - OR (OR)
→ EX-OR - (XOR) → XOR is also MOD 2 func?
→ Clear Carry (CLRC)
→ Set Carry (STC)
→ Complement Carry (CMC) → If carry 0→1, 1→0
→ Enable Interrupt (EI)
→ Disable Interrupt (DI)

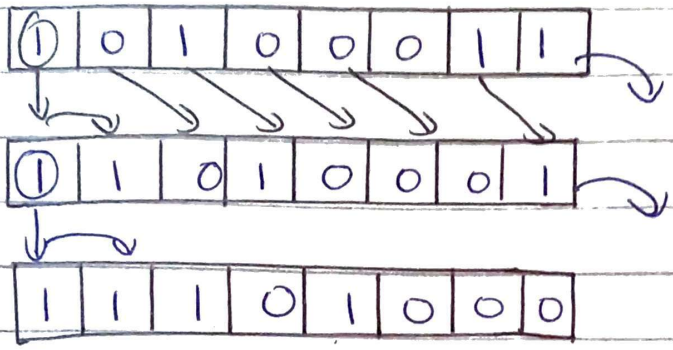
to enable any software interrupts

Shift Instructions



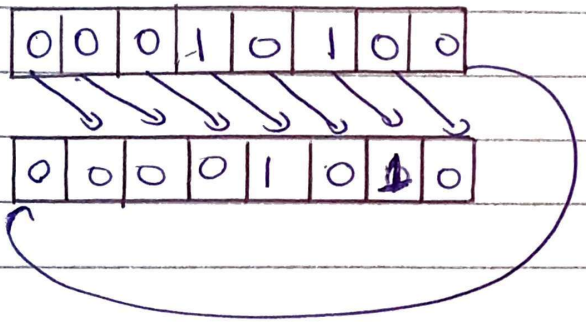
→ Logical shift left
→ Logical shift right
→ Arithmetic shift right
→ Arithmetic shift left
→ Rotate right
→ Rotate ~~re~~ left
→ Rotate right through carry
→ Rotate ~~fo~~ left through carry

Arithmetic Shift Right

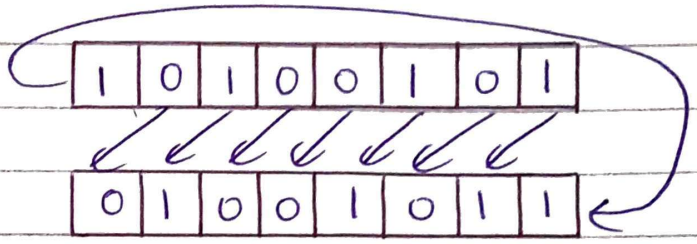


Arithmetic Shift Left same as Logical Shift Left.

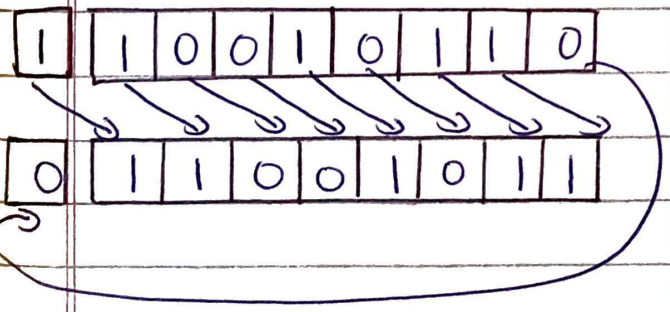
Rotate right



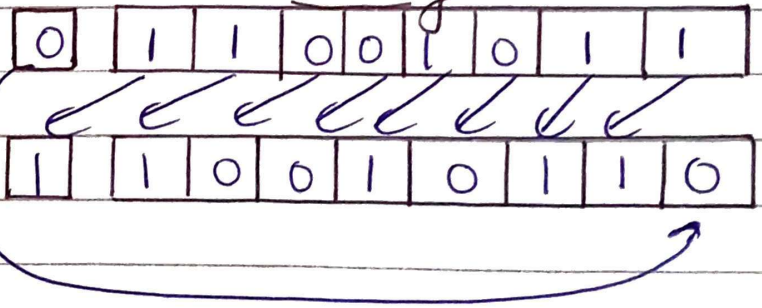
Rotate Left



Rotate right through carry



Rotate Left through carry



For shift instructions:

AH	Opcode	Type	Count	Operand

Program control instructions / Type of control instructions

Branch → Unconditional
Branch → Conditional

Unconditional

JMP 2000

B 3000

Skip → to skip any instruction by skipping the address of that instruction in PC.

CALL 3000

Conditional

BE R1 R2 2000

BNZ R1 2000

Addressing Modes

1) Implied Mode
2) Immediate Mode
3) Register Mode
4) Register Indirect Mode
5) Auto increment or Auto decrement mode
6) Direct Address Mode
7) Indirect Address Mode
8) Relative Address Mode
9) Indexed Address Mode
10) Base register Addressing Mode

OPCODE	OPERAND

1) Implied Mode / Implicit Mode

Operand is specified implicitly in the definition of instruction.

→ Used for zero address and one address instructions.

one address

E.g. INCA (AC ← AC+1)

E.g. CLA

↳ Complement accumulator

Eg. CRC

↳ clear carry

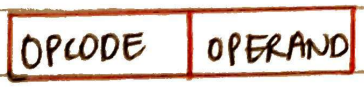
Zero address (for stack)

E.g. Add → Pop last two values and add them, then push them on stack.

2) Immediate mode

→ Operand is directly provided as constant.

→ No computation required to calculate EA



Ex- Add R1, #3 (R1 ← R1+3)

→ Range of operand depends on the length of address field.

3) Register mode

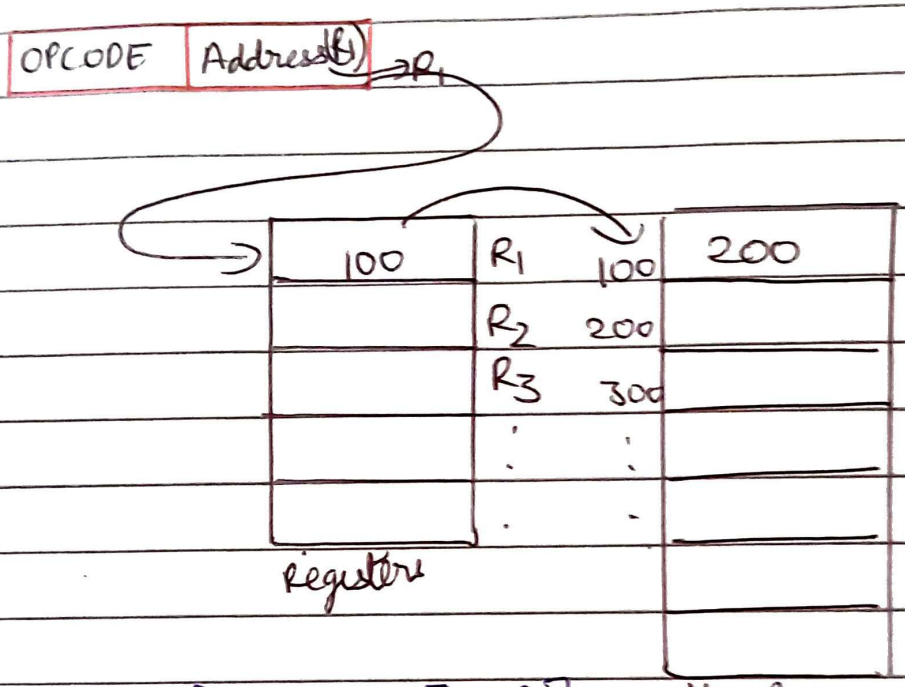
→ Operand is present in the register.

→ Register no is present in the instruction.

Ex- LD R1 (ACC ← R1)

4) Register Indirect Mode

→ Register contains address of operand rather than the operand itself.

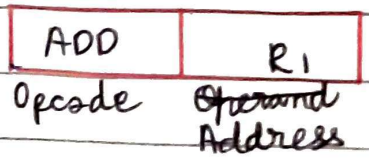


Eg. LD (R1) [AC ← M[CR1]]

Eg. ADD R1, (R2) [R1 ← R1 + M[R2]]

5) Auto increment or Auto decrement Addressing Mode

→ Special case of register indirect addressing Mode.

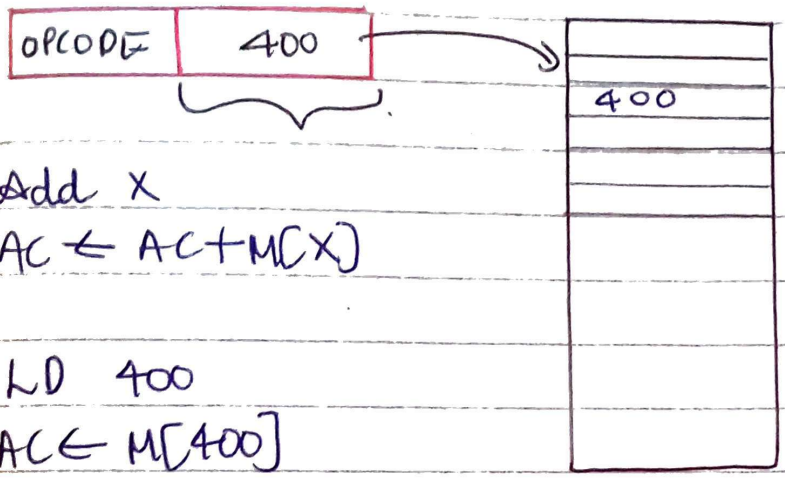


→ By using clock pulse, we increase value of registers.

PAGE NO.
DATE: / /

Direct Addressing Mode (Absolute addressing mode)

→ Actual address is given in instruction.
→ Use to access variables.



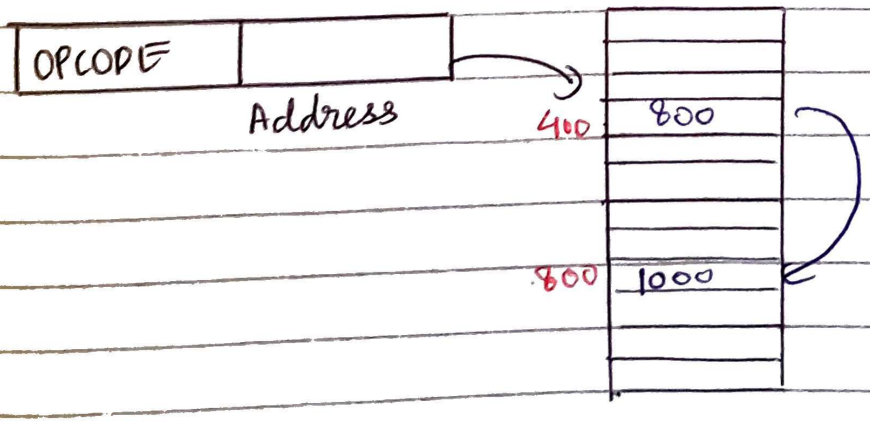
E.g. Add X
AC ← AC + M[X]

E.g. LD 400
AC ← M[400]

Disadvantage: Size of instruction is fixed.

Indirect addressing Mode

→ Used to implement pointers and passing parameters
→ 2 Memory access required.



E.g. ADD X [AC ← AC + M[M[X]]]

EA = M[X]

Eg: int *a = &b;